

What is an Operating System

An **Operating System (OS)** is a special type of software that acts as the backbone of a computer, making it possible for you to use your device effectively. Think of it as the manager of a busy restaurant: it coordinates everything—hardware, software, and users—so that the computer runs smoothly.

Software Abstraction of Hardware

- **What it means:** Hardware includes physical components like the CPU (processor), memory, hard drive, keyboard, and monitor. The OS hides the complex details of how these components work and provides a simpler way for users and programs to interact with them.
- **Example:** When you save a file, you don't need to know how the hard drive stores data or which memory blocks are used. The OS handles all those details for you, making it as simple as clicking "Save."
- **Why it matters:** This abstraction makes computers easier to use, as you don't need to be a hardware expert to operate them.

Interface Between User and Hardware

- **What it means:** The OS acts as a middleman between you (the user) and the computer's hardware. It translates your commands (like clicking an icon or typing) into instructions the hardware can understand.
- **Example:** When you double-click a music file, the OS tells the hardware to open the music player and play the file.
- **Why it matters:** Without an OS, you'd need to write complex code to interact with hardware, which would be impractical for most users.

Set of Utilities to Simplify Application Development/Execution

- **What it means:** The OS provides tools and services that make it easier for developers to create programs (like apps or games) and for those programs to run on your computer.
- **Example:** A game developer doesn't need to write code to manage memory or handle keyboard input from scratch. The OS provides ready-to-use functions for these tasks.
- **Why it matters:** This saves time and effort, allowing developers to focus on creating features rather than dealing with low-level hardware details.

Control Program

- **What it means:** The OS is like a traffic controller, managing how programs and hardware resources (like CPU or memory) are used to avoid conflicts and ensure smooth operation.
- **Example:** If you're watching a video, browsing the web, and downloading a file at the same time, the OS decides which program gets CPU time and when, so nothing crashes.
- **Why it matters:** This ensures your computer can multitask without programs interfering with each other.

Acts Like a Government

- **What it means:** Just as a government sets rules and manages resources for a country, the OS sets rules for how programs and hardware operate, ensuring fairness, security, and efficiency.
- **Example:** The OS ensures one program doesn't hog all the memory, just like a government might regulate resources to ensure fair distribution.
- **Why it matters:** This "governance" keeps the computer system organized and prevents chaos.



Services of Operating Systems

The OS provides a range of services to make computing easier for users and programs. These services are like the utilities in a city—things like water, electricity, and roads—that make life functional and convenient.

User Interface

- **What it means:** The way you interact with the computer, such as clicking icons or typing commands.
- **Types:**
 - **Graphical User Interface (GUI):** Uses visuals like windows, buttons, and icons (e.g., Windows desktop or macOS).
 - **Command-Line Interface (CLI):** Uses text commands (e.g., typing `dir` in Windows Command Prompt to list files).
- **Example:** Clicking the Start menu in Windows or typing `ls` in a Linux terminal.
- **Why it matters:** The user interface makes the computer accessible to everyone, not just tech experts.

Program Execution

- **What it means:** The OS loads and runs programs (like a web browser or a game) by allocating the necessary resources, such as memory and CPU time.
- **Example:** When you open Microsoft Word, the OS loads it into memory and ensures it runs smoothly.
- **Why it matters:** Without this service, you couldn't run apps, as they wouldn't know how to access the computer's resources.

I/O Operation

- **What it means:** Input/Output (I/O) operations involve managing communication between the computer and devices like keyboards, mice, printers, or external drives.
- **Example:** When you print a document, the OS sends the data to the printer and ensures it prints correctly.
- **Why it matters:** The OS simplifies these interactions, so programs don't need to

handle the complex details of each device.

File-System Manipulation

- **What it means:** The OS manages how files and folders are stored, organized, and accessed on storage devices like hard drives or USBs.
- **Example:** When you create, delete, or rename a file, the OS updates the file system to reflect those changes.
- **Why it matters:** This keeps your data organized and accessible, like a librarian managing books in a library.

Communication (Inter-Process Communication)

- **What it means:** The OS allows different programs (or parts of a program) to share data or coordinate tasks.
- **Example:** Copying text from a web browser and pasting it into a text editor involves the OS enabling communication between the two programs.
- **Why it matters:** This ensures programs can work together seamlessly, enhancing functionality.

Error Detection

- **What it means:** The OS monitors the system for errors (like hardware failures or software crashes) and tries to handle them to keep the system running.
- **Example:** If a program tries to use too much memory, the OS might close it to prevent a system crash, showing an error message.
- **Why it matters:** This keeps the computer stable and prevents small issues from becoming big problems.

Resource Allocation

- **What it means:** The OS decides how to distribute resources like CPU time, memory,

and storage among running programs.

- **Example:** If you're running a video editor and a web browser, the OS ensures both get enough CPU power to function without slowing down.
- **Why it matters:** Fair resource allocation prevents one program from monopolizing the system, ensuring smooth multitasking.

Accounting

- **What it means:** The OS tracks how resources are used by programs and users, often for monitoring or billing purposes.
- **Example:** In a shared computer system (like a university server), the OS might log how much CPU time each user consumes.
- **Why it matters:** This helps administrators manage system usage and ensures fairness.

Protection & Security

- **What it means:** The OS ensures that programs, users, and data are protected from unauthorized access or malicious activity.
- **Example:** Requiring a password to log in or preventing one user's program from accessing another user's files.
- **Why it matters:** This keeps your data safe and ensures the system operates securely.

»

Goals of Operating Systems

The OS is designed with specific goals to make computing effective and user-friendly. Think of these as the guiding principles for how an OS should behave.

Convenience (User-Friendly)

- **What it means:** The OS should be easy to use, even for non-experts.
- **Example:** A GUI like Windows makes it simple to open apps by clicking icons instead of typing complex commands.
- **Why it matters:** Convenience ensures anyone can use a computer, not just programmers.

Efficiency

- **What it means:** The OS should use resources (like CPU, memory, and storage) effectively to maximize performance.
- **Example:** The OS schedules tasks so that the CPU is rarely idle, keeping your computer fast.
- **Why it matters:** Efficiency means faster performance and less wasted resources.

Portability

- **What it means:** The OS should work on different types of hardware or be easily adapted to new devices.
- **Example:** Linux runs on everything from laptops to servers to embedded devices like smart TVs.
- **Why it matters:** Portability allows the OS to be versatile and widely used across devices.

Reliability

- **What it means:** The OS should work consistently without crashing or losing data.
- **Example:** Windows automatically saves recovery points so you can restore your system if it crashes.
- **Why it matters:** Reliability ensures you can trust the OS to keep your work safe.

Scalability

- **What it means:** The OS should handle increasing workloads or users without slowing down.
- **Example:** A server OS like Linux can manage thousands of users accessing a website simultaneously.
- **Why it matters:** Scalability ensures the OS can grow with demand, like in cloud computing.

Robustness

- **What it means:** The OS should be resilient to errors, crashes, or attacks and recover quickly.
- **Example:** If a program crashes, the OS isolates the issue so other programs keep running.
- **Why it matters:** Robustness keeps the system stable under challenging conditions.



Parts of an Operating System

The OS is made up of two main components: the **Shell** and the **Kernel**. These work together to make the computer functional.

Shell - Kind of an Interface

- **What it means:** The shell is the part of the OS you interact with directly. It's like the front desk of a hotel, where you make requests.
- **Types:**
 - **Graphical User Interface (GUI):** A visual interface with windows, icons, and menus (e.g., the Windows desktop or macOS Finder).
 - **Command-Line Interface (CLI):** A text-based interface where you type commands (e.g., Linux terminal or Windows Command Prompt).

- **Example:** Clicking a folder icon (GUI) or typing `mkdir newfolder` (CLI) to create a new folder.
- **Why it matters:** The shell makes it easy for users to communicate with the OS, whether they prefer visuals or text.

Kernel - All Functionalities of OS

- **What it means:** The kernel is the core of the OS, handling all the critical tasks like managing hardware, memory, and processes. It's like the engine of a car, doing the heavy lifting behind the scenes.
- **Example:** When you open a program, the kernel allocates memory and CPU time to run it.
- **Why it matters:** The kernel ensures the OS can perform its essential functions, making everything else possible.



System Call

- **What it means:** A **system call** is a way for a program to request a service from the OS. It's like a customer placing an order at a restaurant—the program asks the OS to do something it can't do on its own, like reading a file or sending data to a printer.
- **How it works:** Programs use specific functions (system calls) to communicate with the OS. The OS then performs the requested task and returns the result.
- **Example:** When a text editor saves a file, it uses a system call to ask the OS to write data to the hard drive.
- **Why it matters:** System calls allow programs to access powerful OS services safely, without directly controlling the hardware (which could cause errors or crashes).
- **Common system calls:**
 - `open()` : To access a file.
 - `read()` : To read data from a file or device.
 - `write()` : To write data to a file or device.
 - `fork()` : To create a new process (a running program).

- Types Of System Call

System Call Type	Windows	Unix
Process Control	CreateProcess() , ExitProcess() , WaitForSingleObject()	fork() , exit() , wait()
File Reading	ReadFile() , WriteFile() , CreateFile() , CloseHandle()	read() , write() , open() , close()
Device Management	ReadFile() , WriteFile() , SetConsoleMode() , GetConsoleMode()	ioctl() , read() , write()
Information Maintenance	GetCurrentProcessId() , GetProcessTimes() , GetSystemTime() , SetSystemTime()	getpid() , time() , gettimeofday() , chmod()
Communication	CreatePipe() , CreateMailSlot() , CreateNamedPipe() , ConnectNamedPipe()	pipe() , shmget() , msgget() , socket()
Protection	SetFileSecurity() , GetFileSecurity() , SetAcl()	chmod() , chown() , umask()



Dual Mode of Operation

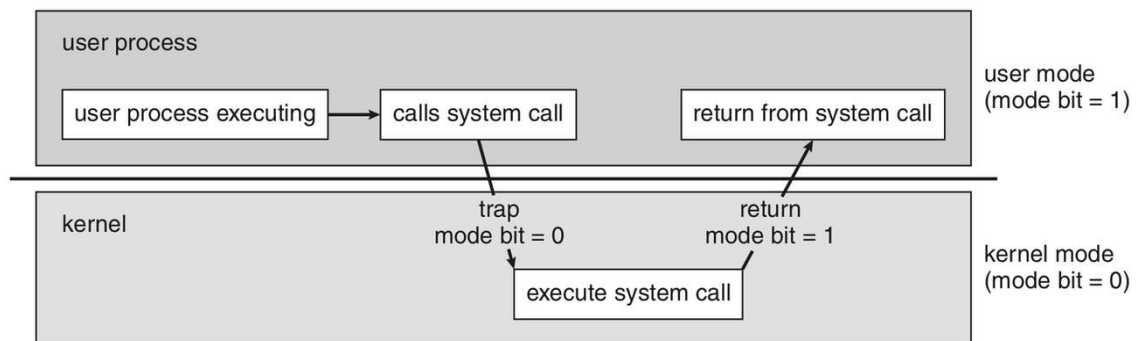
The OS operates in two modes to ensure protection and prevent programs from causing harm. This is like having different levels of access in a building—regular visitors vs. security staff.

Used to Implement Protection

- **What it means:** The dual mode prevents programs from accessing sensitive parts of the system (like hardware or other programs' data) without permission, ensuring stability and security.
- **Why it matters:** Without protection, a faulty or malicious program could crash the system or steal data.

Two Modes

- **User Mode (Mode Bit = 1):**
 - **What it means:** In user mode, programs (like your web browser or games) run with limited privileges. They can only access their own data and must use system calls to request OS services.
 - **Example:** When you open a text editor, it runs in user mode and can't directly control the hard drive or other programs' memory.
 - **Why it matters:** User mode keeps programs safe and isolated, preventing them from causing system-wide problems.
- **Kernel/System/Supervisor/Privileged Mode (Mode Bit = 0):**
 - **What it means:** In kernel mode, the OS has full control over the hardware and system resources. Only the OS itself runs in this mode.
 - **Example:** When the OS writes a file to the hard drive or manages CPU scheduling, it operates in kernel mode.
 - **Why it matters:** Kernel mode allows the OS to perform critical tasks securely and efficiently.



Transition from User to Kernel Mode with an Example

- **What it means:** A transition occurs when a program in user mode needs to perform a task that requires kernel privileges, so it makes a system call, and the OS switches to kernel mode to handle it.
- **How it works:**
 1. A program running in user mode needs to do something privileged, like reading a file.
 2. It makes a **system call** (e.g., `read()`), which triggers a special instruction called a **trap** or **software interrupt**.

3. The CPU switches the mode bit from 1 (user mode) to 0 (kernel mode).
4. The OS's kernel takes over, performs the requested task (e.g., accessing the hard drive), and returns the result.
5. The CPU switches back to user mode (mode bit = 1), and the program continues running.

- **Example:**

- **Scenario:** You're using a text editor to open a file called `notes.txt`.
- **Step 1:** The text editor (running in user mode) wants to read the file but can't directly access the hard drive.
- **Step 2:** The editor makes a system call, `read("notes.txt")`, which sends a request to the OS.
- **Step 3:** The CPU detects the system call and switches to kernel mode (mode bit = 0).
- **Step 4:** The OS kernel accesses the hard drive, reads the contents of `notes.txt`, and stores the data in memory.
- **Step 5:** The kernel returns the data to the text editor and switches back to user mode (mode bit = 1).
- **Step 6:** The text editor displays the file's contents on your screen.
- **Why it matters:** This transition ensures that only the OS (in kernel mode) can perform sensitive tasks, keeping the system secure and stable. If the text editor tried to access the hard drive directly in user mode, it could cause errors or even crash the system.